Содержание

3 Задача об упаковке в контейнеры **4**
3.1 Алгоритм с линейной сложностью . . . 4
3.2 Жадные алгоритмы . . . 5
3.3 Линейная релаксация . . . 7
3.4 Алгоритмы с аддитивной ошибкой . . . 7
3.5 Улучшенная нижняя граница . . . 8

3 Задача об упаковке в контейнеры

В наиболее простой формулировке задача имеет следующий вид:

Имеется n объектов с объёмами v_i, $i = 1, \ldots, n$, а также ёмкости (контейнеры) с вместимостью $\overline{v} \geq \max_i v_i$. Необходимо каждому объекту i сопоставить ровно одну ёмкость k_i таким образом, что

$$\sum_{i:k_i=k} v_i \leq \overline{v} \qquad \forall\ k,$$

т.е., чтобы суммарный объём объектов в любой из ёмкостей не превосходил ограничения сверху $\overline{v}$. Из всех возможных комбинаций следует найти ту, которая использует наименьшее количество ёмкостей.

Очевидным приложением задачи является проблема наполнения контейнеров / машин / вагонов / самолётов и т.д. при перевозке грузов.

В постановке мы предполагаем, что ни один объект не имеет объёма, превосходящего объём контейнеров.

Задача об упаковке в контейнеры НП-сложная, более того, сложно аппроксимировать решение с мультипликативным фактором $\alpha < \frac{3}{2}$. А именно, пусть $\bar{v} = \frac{1}{2}\sum_{i=1}^{n} v_i$. Тогда оптимальное значение равно 2 тогда и только тогда, когда множество объектов делится на два дизъюнктных множества с одинаковым суммарным объёмом. Установить справедливость этого условия эквивалентно задаче о сумме подмножества (subset sum), которая также сложная. С другой стороны, минимальное количество контейнеров не больше 3-х, так как при распределении по 4 контейнерам существует пара контейнеров с суммарным объёмом объектов, не превосходящим $\bar{v}$, и их содержимое можно объединить в одном. Любой алгоритм, аппроксимирующий оптимальное решение с множителем, строго меньшим, чем $\frac{3}{2}$, таким образом, решает задачу о сумме подмножества.

3.1 Алгоритм с линейной сложностью

Самый простой алгоритм обходится $n-1$ сравнениями объёма объекта с остаточным объёмом в контейнере. Алгоритм *следующий подходящий* (next fit) рассматривает объекты без предварительного упорядочивания, и кладёт объект в текущий контейнер, если он туда ещё вмещается, и в следующий (на этот момент ещё пустой) в противном случае. Так как объём объектов предполагается не превосходящим объёма контейнеров, во втором случае сравнения делать не надо, но всё равно нужно посчитать остаточный объём.

Заметим, что в итоге сумма объёмов объектов, оказавшихся в соседних задействованных контейнерах, всегда строго превосходит объём одного контейнера, поскольку первый объект, положенный в следующий контейнер, не уместился в предыдущий. С другой стороны, невозможно уложить все объекты в количество контейнеров, суммарный объём которых строго меньше суммарного объёма объектов, откуда получаем границу $N_{opt} \geq \lceil \frac{\sum_{i=1}^{n} v_i}{\overline{v}} \rceil$.

Пусть $\hat{N}$ — количество ёмкостей, задействованных алгоритмом next fit, а $V_1, \ldots, V_{\hat{N}}$ — суммарные объёмы объектов в этих ёмкостях. Тогда $V_k + V_{k+1} > \overline{v}$ для всех $k = 1, \ldots, \hat{N}-1$, откуда получаем

$$V_1 + 2\sum_{k=2}^{\hat{N}-1} V_k + V_{\hat{N}} > (\hat{N}-1)\overline{v}, \qquad \frac{\sum_{k=1}^{\hat{N}} V_k}{\overline{v}} > \frac{\hat{N}-1}{2}$$

в случае $\hat{N} > 2$ (в случае $\hat{N} \leq 2$ алгоритм, очевидно, выдаёт оптимальное решение). Комбинируя с нижней оценкой на N_{opt}, получаем $N_{opt} > \frac{\hat{N}-1}{2}$, откуда $N_{opt} \geq \frac{\hat{N}}{2}$. Поэтому алгоритм выдаст решение, в котором используется не более, чем вдвое больше контейнеров, чем в оптимальной упаковке.

3.2 Жадные алгоритмы

Относительно простыми модификациями фактор 2 можно строго улучшить. Рассмотрим два похожих алгоритма.

Задачу об упаковке в ёмкости (контейнеры) можно приближённо решить следующим *жадным* алгоритмом *первая подходящяя после сортировки* (first fit decreasing, FFD):

- отсортировать объекты по объёмам v_i в убывающем порядке
- пронумеровать ёмкости
- в цикле по объектам, каждому объекту сопоставить с соответствие *первую* ёмкость, в которую объект войдёт без нарушения ограничения $\overline{v}$ на суммарный объём

В частности, если текущий объект не умещается ни в одну начатую ёмкость, он кладётся в пустую на данный момент ёмкость. Пусть N_{opt} — оптимальное значение функционала, т.е., минимальное число ёмкостей, нужное для упаковки всех объектов, и $\hat{N}$ — число ёмкостей, которое необходимо при упаковке жадным алгоритмом. Можно показать [1], что

$$\hat{N} \leq \left\lfloor \frac{11}{9} N_{opt} + \frac{2}{3} \right\rfloor. \tag{1}$$

Таким образом, асимпототически качество решения улучшается, множитель понижается до $\frac{11}{9}$.

Есть следующая вариация алгоритма, ищущая ёмкость, в которой останется минимальный остаточный объём после упаковки текущего предмета (*лучшая подходящяя после сортировки*, best fit decreasing, BFD):

- отсортировать объекты по объёмам v_i в убывающем порядке
- пронумеровать ёмкости
- в цикле по объектам, каждому объекту сопоставить с соответствие первую из ёмкостей с *минимальным* остаточным объёмом, в которую объект войдёт без нарушения ограничения $\overline{v}$ на суммарный объём

Оценка оптимальная, как показывает следующий пример.

Пример: Пусть $k \in \mathbb{N}_+$, и имеется по $6k+4$ объектов с объёмами u, v, w, соответственно, и $12k+8$ объектов с объёмом z. При этом положительные величины $u > v > w > z$ удовлетворяют неравенствам

$$2u > \overline{v}, \quad \overline{v} \geq u + v, \quad 3w + z > \overline{v} \geq 3w, \quad u + w + z = \overline{v}, \quad 2(v+z) = \overline{v}, \quad 5z > \overline{v}.$$

Тогда обе версии жадного алгоритма построят следующее решение:

- объекты с объёмом u попадут в ёмкости $1, \ldots, 6k+4$, так как $2u > \overline{v}$
- объекты с объёмом v также попадут в ёмкости $1, \ldots, 6k+4$, так как $u + v \leq \overline{v} = u + w + z < u + 2v$
- объекты с объёмом w попадут в ёмкости $6k+5, \ldots, 8k+5$ по три объекта в каждую, так как $u+v+w > u + w + z = \overline{v}$, $3w \leq \overline{v} < 3w + z < 4w$
- последний объект с объёмом w попадёт в ёмкость $8k+6$ вместе с тремя объектами с объёмом z, так как $w + 3z \leq \overline{v} < w + 4z$
- объекты с объёмом z попадут в ёмкости $8k+7, \ldots, 11k+7$ по 4 объекта в каждую, так как $u+v+z > u + w + z = \overline{v}$, $3w + z > \overline{v}$, $4z \leq \overline{v} < 5z$

- последний объект с объёмом z попадёт в ёмкость $11k + 8$

Значение этого решения равно $11k + 8$.

Если представить $u = \overline{v} - w - z$, $v = \frac{\overline{v}}{2} - z$ как функции от w, z, то система условий на параметры w, z сводится к строгим неравенствам

$$5z > \overline{v}, \qquad 3w + z > \overline{v}, \qquad w + z < \frac{\overline{v}}{2}.$$

Им удовлетворяет, например, точка

$$u = 31, \quad v = 17, \quad w = 16, \quad z = 13, \quad \overline{v} = 60.$$

Однако, оптимально было бы совместить объекты с объёмом u с объектами с объёмом w, так как тогда в каждую из соответствующих ёмкостей можно дополнительно положить ещё один объект с объёмом z. Оставшиеся $6k + 4$ объектов с объёмом v и $6k + 4$ объектов с объёмом z можно разложить по $3k + 2$ ёмкостям, по набору из 2 объектов каждого вида. Так как все использованные ёмкости в этом решении полные, улучшить его невозможно. Таким образом, оптимальное значение задачи равно $N_{opt} = 9k + 6$.

Сравнив с результатом $\hat{N} = 11k + 8$ алгоритма, получаем

$$\hat{N} = \frac{11}{9} N_{opt} + \frac{2}{3},$$

т.е., оценка (1) не улучшаемая.

Пример показывает, что зазор между значениями оптимального решения и решения, выданного эвристикой, является следствием неправильной упаковки больших объектов. Более тщательный анализ возможностей скомбинировать большие объекты позволяет дальше снизить гарантированное соотношение между этими значениями. Например, алгоритм, предложенный Йе и Зангом (Yue, Zhang) в [5], отдельно рассматривает объекты с объёмами в интервалах $(0, \frac{1-v_{\min}}{5}]$, $(\frac{1-v_{\min}}{5}, \frac{1}{3}]$, $(\frac{1}{3}, \frac{1}{2}]$, $(\frac{1}{2}, 1]$, где $v_{\min} = \min_i v_i$, и достигает фактора аппроксимации $\frac{71}{60}$. Алгоритм действует следующим образом:

1. отсортировать объекты по объёмам v_i в убывающем порядке
2. разделить их на классы A, B, C, D согласно вышеуказанным интервалам объёмов
3. для $i = 1, \ldots, |A|$, положить объект i в ёмкость i
4. в цикле по $i = |A|, \ldots, 1$, если это возможно, упаковать два объекта из C в ёмкость i следующим образом:
   (a) положить в ёмкость i наименьший оставшийся объект из C
   (b) доложить наибольший из оставшихся объектов из C, которые ещё вмещаются
   (c) заменить ранее положенный наименьший объект на наибольший из оставшихся объектов из C, которые ещё вмещаются
5. в цикле по оставшимся объектам, каждому объекту сопоставить с соответствие первую из ёмкостей, в которую объект войдёт без нарушения ограничения $\overline{v}$ на суммарный объём

Значение $\hat{N}$ решения, выданного алгоритмом, соотносится с оптимальным значением N_{opt} следующим образом:

$$\hat{N} \leq \left\lfloor \frac{71}{60} N_{opt} \right\rfloor + 1.$$

Основным методом доказательства этого и предыдущих результатов по ограничению зазора между оптимальным значением и значением решения, выданного алгоритмом, является т.н. *метод минимального контрпримера*. Суть этого метода состоит в следующем.

Пусть мы хотим доказать неравенство вида

$$\hat{N} \leq f(N_{opt})$$

для некоторой функции f. Исходя от противного, предположим, что есть контрпример, т.е., набор объёмов $v_1, \ldots, v_n$ и порог $\overline{v}$ такие, что алгоритм выдаёт решение, удовлетворяющее $\hat{N} > f(N_{opt})$. Пусть этот набор — минимальный, т.е., нет такого набора с меньшим числом n. Метод доказательства состоит в том, чтобы показать, что либо этот набор обладает некоторым свойством, либо на его основании можно построить такой же набор с меньшим n. Из этого следует, что свойство должно быть выполнено. В итоге набор таких свойств позволяет свести само существование контрпримера к противоречию [5].

3.3 Линейная релаксация

Задачу об упаковке можно следующим образом записать как смешанно-целочисленную линейную программу (задачу ЦЛП):

$$\min_{X,y} \langle \mathbf{1}, y \rangle : \qquad Xv \leq \overline{v}y, \quad X^T\mathbf{1} = \mathbf{1}, \quad y \in \{0,1\}^n,\ X \in \{0,1\}^{n\times n}.$$

Здесь $v \in \mathbb{R}^n_{++}$ — вектор объёмов, бинарная переменная y_i определяет, используется ли ёмкость i, а бинарная переменная X_{ij} определяет, попадает ли объект j в ёмкость i. Вектор (или более общо, объект) из всех единиц обозначен через $\mathbf{1}$.

Условия $X^T\mathbf{1} = \mathbf{1}$ являются ограничениями т.н. *типа SOS1* (special ordered set type 1). Они расшифровываются в виде $\sum_{i=1}^n X_{ij} = 1$ для любого j, т.е., из бинарных переменных $X_{1j}, \ldots, X_{nj}$ ровно одна принимает значение 1, а остальные — 0. Такие условия сильно ограничивают число комбинаций значений бинарных переменных, и поэтому в алгоритмах решения задач ЦЛП они учитываются отдельно.

Линейная релаксация задачи получается заменой включения переменных в дискретное множество $\{0,1\}$ на включение в его выпуклую оболочку $[0,1]$. Она имеет очевидное решение

$$X = \frac{\mathbf{1}}{n}, \quad y = \frac{\langle \mathbf{1}, v\rangle}{n\overline{v}} \cdot \mathbf{1}$$

с оптимальным значением $\frac{\langle \mathbf{1}, v\rangle}{\overline{v}}$. Таким образом, величина $\lceil \frac{\langle \mathbf{1}, v\rangle}{\overline{v}} \rceil$ является нижней границей оптимального значения задачи об упаковке.

Так как задача симметрична по отношению к перестановкам элементов y с одновременной перестановкой строк матрицы X, то можно добавить дополнительные ограничения $y_i \geq y_{i+1}$ для $i = 1, \ldots, n-1$ в формулировку ЦЛП. Это облегчит работу алгоритма типа ветвей и границ для решения этой задачи.

Пусть N_{lin} значение линейной релаксации. Тогда

$$N_{opt} \leq 2N_{lin},$$

т.е., нижняя граница, задаваемая релаксацией, аппроксимирует оптимальное количество контейнеров с точностью до множителя 2. Действительно, в оптимальном решении не может быть двух ёмкостей, сумма объёмов объектов в которых не превосходит $\overline{v}$, так как иначе объекты в этих ёмкостях можно было бы объединить в одной ёмкости. Просуммировав соответствующие неравенства по всем парам ёмкостей, получим $\langle \mathbf{1}, v\rangle = \sum_{i=1}^n v_i \geq \frac{N_{opt}\overline{v}}{2}$.

3.4 Алгоритмы с аддитивной ошибкой

Асимптотическое соотношение $\frac{71}{60}$ можно сколь угодно близко придвинуть к 1, при этом оставаясь в рамках полиномиально сложных алгоритмов. Идея этих алгоритмов состоит в том, чтобы округлить объёмы объектов таким образом, чтобы они были элементом некоторого множества чисел контролируемой мощности m. Тогда будет существовать определённое количество M типов заполнения ёмкости, задаваемых вектором из $\mathbb{N}^m$. Распределение объектов по ёмкостям также задаётся целым вектором из $\mathbb{N}^M$, определяющим, сколько ёмкостей каждого типа производит заполнение.

Более конкретно, можно выделить следующие шаги:

- объекты маленького объёма во время работы основной фазы алгоритма не рассматриваются, а добавляются в конце в оставшиеся дыры и/или в новые ёмкости; в силу малого суммарного объёма количество дополнительных ёмкостей $o(N_{opt})$

- остальные объекты сортируются по объёму и объединяются в группы, объём объекта в группе округляется вверх до максимального объёма в этой группе
- задача находждения оптимального количества каждого типа заполнения решается через линейную релаксацию, которая производит решение с малым количеством задействованных типов — не более, чем число групп объектов; решается двойственная ЛП с помощью отделяющего оракула
- объекты, попавшие в дробные части количеств типов заполнения, изымаются и пакуются отдельно жадным алгоритмом или (если это лучше), дробное количество типов округляется вверх до следующего целого

В работе [2] для произвольного $\epsilon > 0$ строится алгоритм сложности $C(\epsilon) + O(n \log \epsilon^{-1})$, который гарантированно достигает $\hat{N} \leq (1+\epsilon)N_{opt} + O(1)$ ёмкостей.

В работе [4] предлагается семейство алгоритмов. При определённых настройках гарантированно достигается $\hat{N} \leq N_{opt} + O(\log^2 N_{opt})$ за полиномиальное время.

В работе [3] гарантия улучшается до $\hat{N} \leq N_{opt} + O(\log N_{opt})$ за полиномиальное время.

3.5 Улучшенная нижняя граница

Относительно легко можно получить нижнюю оценку, которая аппроксимирует оптимальное значение с точностью до множителя $\frac{3}{2}$. Для этого поделим множество объектов на два подмножества

$$I_1 = \left\{ i \mid v_i > \frac{\overline{v}}{2} \right\}, \qquad I_2 = \left\{ i \mid v_i \leq \frac{\overline{v}}{2} \right\}.$$

Ясно, что ёмкость не может вместить два объекта из подмножества I_1. Поэтому $N_{opt} \geq |I_1|$. Тогда получаем границу

$$N_{low} = \max\{|I_1|, N_{lin}\}.$$

Покажем, что

$$N_{opt} \leq \left\lfloor \frac{3}{2} N_{low} \right\rfloor.$$

Действительно, построим (недопустимое) заполнение, использующее N_{low} ёмкостей. На первом шаге отведём для каждого объекта из I_1 отдельную ёмкость. Далее начнём заполнять оставшееся пространство в этих ёмкостях, а также новые ёмкости, если понадобится, объектами из I_2, которые будем отбирать один за другим. Но при этом мы разрешаем деление объекта на две или более ёмкостей, т.е., если текущий объект не вмещается в текущую ёмкость, то мы её заполняем доверху частью объёма объекта, а оставшуюся часть вкладваем в следующую или следующие ёмкости. В итоге будет использовано N_{low} ёмкостей, но часть объектов окажется разделёнными на несколько ёмкостей. Для доказательства качества оценки достаточно использовать два факта:

- разделёнными оказываются не более, чем $N_{low} - 1$ объектов
- каждый из разделённых объектов имеет объём, не превосходящий $\frac{\overline{v}}{2}$

Поэтому разделённые объекты можно упаковать в $\leq \lceil \frac{N_{low}-1}{2} \rceil = \lfloor \frac{N_{low}}{2} \rfloor$ новых ёмкостей, по два объекта минимум в каждую, кроме последней, которая может содержать один объект. В итоге получим допустимое заполнение, использующее не более

$$N_{low} + \left\lfloor \frac{N_{low}}{2} \right\rfloor = \left\lfloor \frac{3}{2} N_{low} \right\rfloor$$

ёмкостей.

Оценку можно ещё улучшить, выделив три подмножества объектов. Фиксируем $\alpha \in (0, \frac{1}{2})$ и определим

$$I_1 = \{i \mid v_i \geq (1-\alpha)\overline{v}\}, \qquad I_2 = \left\{ i \mid \frac{1}{2}\overline{v} < v_i < (1-\alpha)\overline{v} \right\}, \qquad I_3 = \left\{ i \mid \alpha\overline{v} < v_i \leq \frac{1}{2}\overline{v} \right\}.$$

Объекты с объёмом не более, чем $\alpha\overline{v}$, игнорируются. Ясно, что каждому объекту из объединения $I_1 \cup I_2$ нужна своя отдельная ёмкость. С другой стороны, объект из подмножества I_3 не уместится в ёмкости, где уже лежит объект из подмножества I_1. Отсюда получаем оценку

$$N_{opt} \geq \max\left(N_{lin}, |I_1| + |I_2| + \max\left(0, \frac{\sum_{i\in I_3} v_i - \sum_{i\in I_2}(\overline{v} - v_i)}{\overline{v}}\right)\right).$$

При $\alpha = 0$ получаем предыдущую нижнюю оценку N_{low}. Ясно, что как функция от α эта нижняя граница кусочно-постоянная, так как α влияет только на состав подмножеств I_1, I_2, I_3. Поэтому для вычисления максимума по α достаточно проверить конечное число значений для α, не превосходящее количество объектов.

Список литературы

[1] György Dósa. The tight bound of first fit decreasing bin-packing algorithm is $ffd(i) \leq 11/9opt(i) + 6/9$. In Chen Bo, Mike Paterson, and Zhang Guochuan, editors, *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies. First International Symposium, ESCAPE 2007*, volume 4614 of *Lecture Notes in Computer Science*, pages 1–11. Springer, Hangzhou, China, 2007.

[2] W. Fernandez de la Vega and G.S. Luecker. Bin packing can be solved within $1 + \epsilon$ in linear time. *Combinatorica*, 1(4):349–355, 1981.

[3] Rebecca Hoberg and Thomas Rothvoss. A logarithmic additive integrality gap for bin packing. In *Proc. 2017 Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2616–2625, 2017.

[4] Narendra Karmarkar and Richard M. Karp. An efficient approximation scheme for the one-dimensional bin-packing problem. In *23rd Annual Symp. on Foundations of Computer Science*, pages 312–320, 1982.

[5] Minyi Yue and Lei Zhang. A simple proof of the inequality $mffd(l) \leq 71/60opt(l) + 1$ for the MFFD bin-packing algorithm. *Acta Mathematicae Applicatae Sinica*, 11(3):318–330, 1995.